RAZOR PAGES ASSIGNMENT DOCUMENT

Razor Pages Implementation in ASP.NET Core

Name: Diksha Chaudhary

Technology: ASP.NET Core Razor Pages

IDE: Visual Studio Code (macOS)

Date: *(26/02/2026)*

---

1. Objective

The objective of this assignment is to understand the working of Razor Pages in ASP.NET Core and to implement a simple application that allows users to add and view items dynamically. This assignment demonstrates page navigation, model binding, handler methods, and Razor syntax.

---

2. Introduction to Razor Pages

Razor Pages is a page-based programming model in ASP.NET Core that simplifies the development of web applications. Each page has:

- A .cshtml file for UI
- A PageModel class for handling logic

This structure helps in separating the presentation from the business logic while keeping related code together.

---

3. Project Structure

The application contains:

- **Model**
  - Item class used to store the item name
- **Pages**
  - Index page → displays the list of items
  - AddItem page → allows user to add a new item

The data is stored in a static in-memory list, so no database is required.

---

4. Model Implementation

A model class named **Item** is created inside the *Models* folder.

It contains a property to store the name of the item.

```
Item.cs 1 ×   Index.cshtml.cs   Index.cshtml   AddItem.cshtml   Ac
Models > Item.cs > ...
1  namespace RazorPagesDemo.Models
2  {
       2 references
3      public class Item
4      {
           2 references
5          public string Name { get; set; }
6      }
7  }
```

---

5. Index Page – Displaying the Item List

The Index page is responsible for displaying all the items using Razor syntax.

The list is rendered dynamically using:

- @model directive
- @foreach loop

This ensures that whenever a new item is added, it appears automatically on the page.

```
Index.cshtml.cs   Index.cshtml ×   AddItem.cshtml
Pages > Index.cshtml
1   @page
2   @model IndexModel
3
4   <h2>Item List</h2>
5
6   <ul>
7   @foreach (var Item item in IndexModel.Items)
8   {
9       <li>@item.Name</li>
10  }
11  </ul>
12
13  <a href="/AddItem">Add New Item</a>
```

---

6. Add Item Page – Adding New Items

The AddItem page contains:

- A form to enter the item name
- BindProperty for model binding
- OnPost() handler method to process form submission

When the form is submitted:

1. The item is added to the list
2. The page redirects back to the Index page
3. The updated list is displayed

Item.cs 1 | Index.cshtml.cs | Index.cshtml | AddItem.cshtml

Pages > AddItem.cshtml

```
@page
@model RazorPagesDemo.Pages.AddItemModel

<h2>Add Item</h2>

<form method="post">
    <div>
        <label asp-for="Name"></label>
        <input asp-for="Name" />
        <span asp-validation-for="Name"></span>
    </div>
    <button type="submit">Add</button>
</form>
```

---

7. Working of Handler Methods

Two handler methods are used:

- **OnGet()** → loads the page
- **OnPost()** → processes form data

The redirection is handled using: return RedirectToPage("Index");

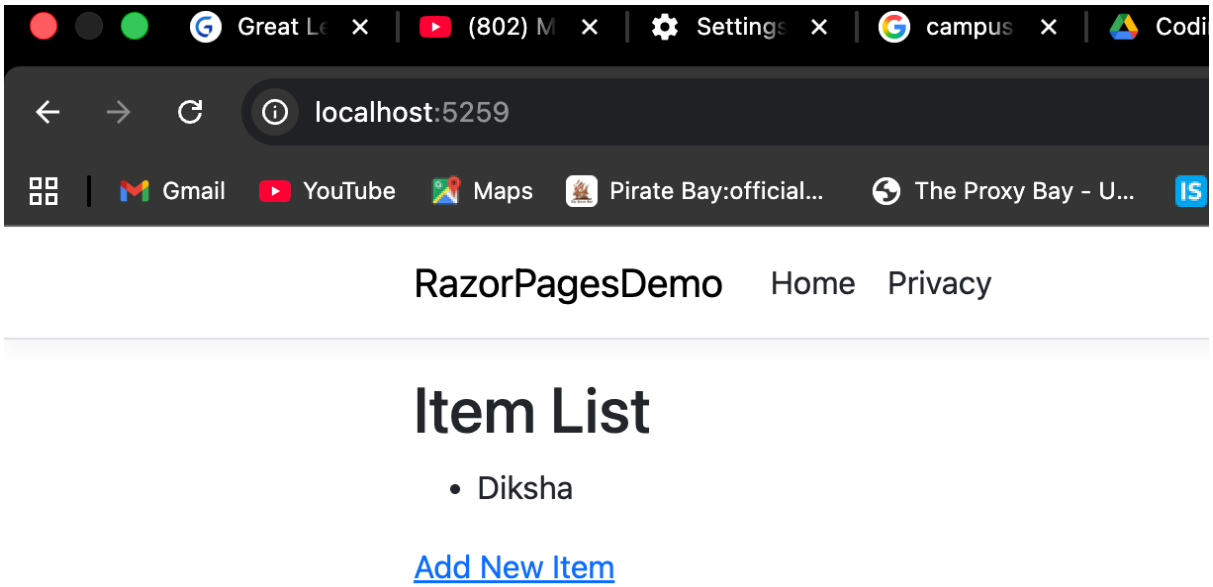
This follows the Post-Redirect-Get pattern.

8. Testing and Output

The application was executed using: dotnet run

The following scenarios were tested:

- Opening the Index page with an empty list
- Navigating to the Add Item page
- Adding a new item
- Redirecting to the Index page
- Displaying the newly added item



9. Result

The Razor Pages application successfully allows users to add items and view them dynamically. The navigation between pages works correctly, and the data binding is performed using the PageModel class.

10. Conclusion

This assignment helped in understanding the Razor Pages architecture and how UI and backend logic are connected using the PageModel. It also demonstrated form handling, model binding, and dynamic page updates without using a database.